

Système distribué d'apprentissage fédéré

INF4218 — Programmation Distribuée

MFENJOU ANAS CHERIF (21T2330)
EDU GUIEDI HERMANN ARNOLD (22T2876)
LANGOUL FADILA MARIAMA MOUNIRA (21T2528)

Université de Yaoundé I
Faculté des Sciences — Département d'Informatique
Master I Systèmes et Réseaux
Année académique 2025–2026

Juin 2026



Plan

- 1 Introduction
- 2 Conception
- 3 Systèmes de nommage
- 4 Coordination distribuée
- 5 Communication et tolérance aux pannes
- 6 Implémentation et validation
- 7 Résultats expérimentaux
- 8 Démonstration
- 9 Conclusion

Cadre du projet

Travail pratique **INF4218 — Programmation Distribuée**

Référence : Van Steen & Tanenbaum, *Distributed Systems*, ch. 1 à 8

Objectif : concevoir un **système distribué d'apprentissage fédéré** (*Federated Learning*) mettant en œuvre l'ensemble des concepts obligatoires du cours.

- Clients : entraînement **local** sur données privées
- Serveur : **agrégation** des gradients → modèle global
- Illustration : coordination, RPC, cohérence temporelle, tolérance aux pannes

Stack : Python 3.14, gRPC/Protobuf, NumPy, pytest

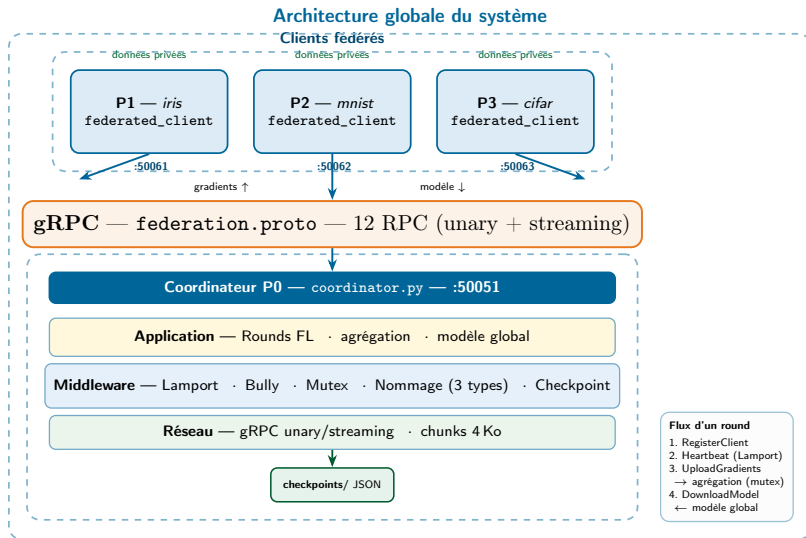
Dépôt : <https://github.com/CherifMfenjou/INF4218-TPSD>

Concepts obligatoires implémentés

Concept	Chapitre Tanenbaum
Architecture client-serveur multi-niveaux	2
Nommage plat / structuré / par attributs	6.2 – 6.4
Horloges logiques de Lamport	5.2.1
Élection de leader (Bully)	5.4.1
Exclusion mutuelle distribuée (Ricart-Agrawala)	5.3.3
Communication RPC / gRPC	4
Tolérance aux pannes (checkpoint / recovery)	8.6.2

41 tests (40 unitaires + 1 intégration) — **tous passent**

Architecture globale du système



Architecture à trois niveaux

Couche Application

Rounds FL, entraînement local, agrégation fédérée

Couche Middleware

Lamport, Bully, mutex, nommage, checkpoint

Couche Réseau (gRPC)

RPC unary + streaming

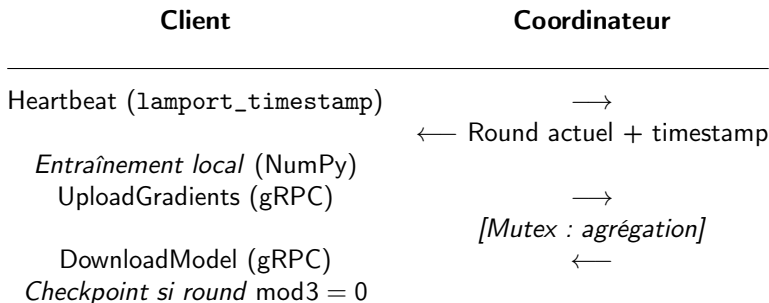
Serveur FederationCoordinator (P0)

- Coordination des rounds et modèle global
- Registres de nommage (3 paradigmes)
- Checkpoints et recovery

Clients FederatedClient (P1, P2, P3...)

- Inscription (RegisterClient)
- Entraînement local et échange de gradients
- Heartbeats périodiques

Flux d'un round d'apprentissage fédéré



- 1 Signal de vie horodaté (Lamport)
- 2 Calcul local des gradients
- 3 Transfert client → serveur ; agrégation en section critique
- 4 Diffusion du modèle global mis à jour

Trois paradigmes de nommage

Nommage plat (UUID)

Identifiant opaque unique — clé primaire pour gradients, heartbeats et checkpoints.

```
af4b9420-9e3e-443e-a2c9-9b8199685f2e
```

Nommage structuré

Chemins hiérarchiques inspirés du DNS / systèmes de fichiers :

```
/federation/clients/client1/models/local
```

Nommage par attributs

Recherche dynamique par paires clé-valeur via QueryByAttributes :

```
{dataset: iris}, {dataset: mnist}, {location: local}
```

Les trois enregistrements sont effectués simultanément à RegisterClient.

Horloges logiques de Lamport

Objectif : ordonner les événements sans horloge physique synchronisée.

- Événement local : $C_i \leftarrow C_i + 1$
- Envoi de message : attacher $ts(m) = C_i$ au message
- Réception : $C_j \leftarrow \max(C_j, ts) + 1$

Intégration dans le système

Chaque message gRPC transporte un `lamport_timestamp`.
Garantit l'ordre causal (*happens-before*) des événements de rounds.

Résultat (`lamport_round_consistency`) :

`ordering_violations = 0`, `consistent = true`,

`final_clock_values = [100, 100, 100, 100, 99]`

Élection Bully et exclusion mutuelle

Bully (élection de leader, ch. 5.4.1)

- ID maximal vivant = coordinateur
- Messages ELECTION / COORDINATOR
- Reprise automatique après panne-crash

Expérience : `bully_election` :
P7 tombe → P6 élu en 0,103 ms ;
`success = true`

Ricart-Agrawala (mutex distribué, ch. 5.3.3)

- Protège `aggregate_round()` (section critique)
- Accès prioritaire : plus petit timestamp Lamport
- Absence de deadlock et de famine

Sans mutex : risque de corruption concurrente de `model_weights`

Contrat : `proto/federation.proto` — service
`FederationCoordinator`, **12 RPC**

Unary (messages légers)

- `RegisterClient`, `Heartbeat`
- `RequestMutualExclusion`,
`ReleaseMutualExclusion`
- `ResolveName`,
`QueryByAttributes`
- `SendElection`,
`AnnounceCoordinator`

Streaming (gros volumes)

- `UploadGradients` (client → serveur)
- `DownloadModel` (serveur → client)
- Chunks de 4 Ko; seuil
`STREAMING_THRESHOLD = 8 Ko`

Démo : gradients de 80 o (10 floats) → unary ; modèles > 64 Ko → streaming recommandé

Tolérance aux pannes

- **Détection** : absence de heartbeat (> 5 s en production ; 2 s en benchmark)
- **Checkpoint** : sauvegarde JSON périodique (intervalle : 3 rounds)
- **État persisté** : `current_round`, `model_weights`, horloge Lamport, `coordinator_id`
- **Recovery** : `_recover_state()` au redémarrage du serveur

Résultat (`fault_tolerance`)

```
recovered_round = 9, crash_detected = true,  
checkpoints_available = [round_3, round_6, round_9], success  
= true
```

Structure du projet

TPSD/

proto/federation.proto	# 12 RPC gRPC
src/naming/	# plat, structuré, attributs
src/coordination/	# lamport, bully, mutex
src/fault_tolerance/	# checkpoint, recovery
src/utils/network.py	# IP locale, adresses client/serveur
src/server/coordinator.py	# serveur coordinateur
src/client/federated_client.py	# client fédéré
src/menu.py	# menu interactif
tests/	# 41 tests (pytest)
experiments/benchmarks.py	# 4 expériences
run_demo.sh / run_menu.sh	# démo auto ou menu interactif

Couverture des tests unitaires

Module	Tests	Couverture
Lamport (lamport.py)	7	tick, send/receive, happens-before
Bully (bully.py)	7	élection, panne, annonce coordinate
Nommage (naming/)	12	plat, structuré, attributs
Mutex (mutual_exclusion.py)	5	accès, conflit, release
Checkpoint (checkpoint.py)	7	save/load, recovery, timeout
Réseau (network.py)	2	IP locale, ports clients distincts
Intégration (test_integration.py)	1	client ↔ serveur gRPC
Total	41	pytest tests/ -v

Exécution

Date : **2026-06-26 22 :50 :49**

Commande : `python experiments/benchmarks.py`

Export : `experiments/results/benchmark_results.json`

Quatre expériences enregistrées dans le fichier JSON :

- 1 `lamport_round_consistency`
- 2 `communication_strategies`
- 3 `bully_election`
- 4 `fault_tolerance`

Exp. 1 — lamport_round_consistency

Protocole : 5 processus, 10 rounds ; vérification $C(\text{réception}) > C(\text{envoi})$.

Champ JSON	Valeur
num_processes	5
num_rounds	10
total_events	250
ordering_violations	0
consistent	true
final_clock_values	[100, 100, 100, 100, 99]

Aucune violation causale ; horloges cohérentes (P0–P3 : 100, P4 : 99).

Exp. 2 — communication_strategies

o	chunks	unary (ms)	stream (ms)	ratio
1 024	1	0,0	0,004	9,83
4 096	1	0,0	0,002	7,94
16 384	4	0,0	0,003	19,63
65 536	16	0,0	0,006	30,12
262 144	64	0,0	0,035	76,30

Overhead streaming : $9,83\times \rightarrow 76,30\times$.

Unary si < 8 Ko (d  mo : 80 o) ; streaming si gros mod  les (> 64 Ko).

Exp. 3 — bully_election

Protocole : 8 processus (P0–P7); panne de P7; élection lancée par P4.

Champ JSON	Valeur
num_processes	8
failed_coordinator	7
new_coordinator	6
election_time_ms	0,103
success	true

P6 (ID maximal vivant) élu coordinateur en 0,103 ms.

Exp. 4 — fault_tolerance

Protocole : 9 rounds ; snapshots rounds 3, 6, 9 ; crash simulé (timeout 2 s).

Champ JSON	Valeur
rounds_before_crash	9
crash_detected	true
recovered_round	9
recovered_weights	$10 \times 0,9$
checkpoints_available	round_3, round_6, round_9
success	true

Reprise exacte au round 9 ; perte max. : 2 rounds entre checkpoints.

Synthèse des résultats (benchmark_results.json)

Expérience	Indicateur	Résultat
lamport_round_consistency	ordering_violations	0 / 250
communication_strategies	ratio max.	76,30× (262 Ko)
bully_election	election_time_ms	0,103 ms; P6
fault_tolerance	recovered_round	9; success
Tests	pytest tests/ -v	41/41

Reproductibilité : `./run_demo.sh` | `./run_menu.sh` | `python experiments/benchmarks.py`

Démonstration du système

Automatique

```
./run_demo.sh
```

Menu interactif

```
./run_menu.sh
```

Serveur, clients, benchmarks, tests, checkpoints, nommage, état réseau

3 clients (iris, mnist, cifar); inscription triple nommage; échanges gRPC horodatés; upload unary (80 o); download streaming

Dépôt : <https://github.com/CherifMfenjou/INF4218-TPSD>

Réseau multi-machines

- Détection IP LAN (get_local_ip)
- Ports clients : 50061, 50062, 50063...
- Serveur : HOST=192.168.1.100
./run_demo.sh
- Client distant : -server
192.168.1.100:50051

Bilan

- Système d'apprentissage fédéré intégrant **l'ensemble** des concepts INF4218
- Architecture modulaire en 3 niveaux, testée et documentée
- Validation expérimentale : Lamport, Bully, gRPC, checkpoint

Limitations

- Coordinateur central (goulot d'étranglement)
- Bully : complexité $O(n^2)$ en messages
- gRPC non chiffré ; pas de tolérance Byzantine
- Entraînement ML simulé (NumPy)

Perspectives : Raft, TLS/mTLS, Secure Aggregation, clients multi-langages

Merci de votre attention

Questions ?



MFENJOU ANAS CHERIF | EDU GUIEDI HERMANN ARNOLD | LANGOUL
FADILA MARIAMA MOUNIRA
INF4218 — UY1